need for waiting. This is a desirable property to have but hard to achieve. It's all too easy to end up writing what's essentially a spin lock.

A second reason to use lock-free data structures is robustness. If a thread dies while holding a lock, that data structure is broken forever. But if a thread dies partway through an operation on a lock-free data structure, nothing is lost except that thread's data; other threads can proceed normally.

The flip side here is that if you can't exclude threads from accessing the data structure, then you must be careful to ensure that the invariants are upheld or choose alternative invariants that can be upheld. Also, you must pay attention to the ordering constraints you impose on the operations. To avoid the undefined behavior associated with a data race, you must use atomic operations for the modifications. But that alone isn't enough; you must ensure that changes become visible to other threads in the correct order. All this means that writing thread-safe data structures without using locks is considerably harder than writing them with locks.

Because there aren't any locks, deadlocks are impossible with lock-free data structures, although there is the possibility of live locks instead. A *live lock* occurs when two threads each try to change the data structure, but for each thread the changes made by the other require the operation to be restarted, so both threads loop and try again. Imagine two people trying to go through a narrow gap. If they both go at once, they get stuck, so they have to come out and try again. Unless someone gets there first (either by agreement, by being quicker, or by sheer luck), the cycle will repeat. As in this simple example, live locks are typically short lived because they depend on the exact scheduling of threads. They therefore sap performance rather than cause long-term problems, but they're still something to watch out for. By definition, wait-free code can't suffer from live lock because there's always an upper limit on the number of steps needed to perform an operation. The flip side here is that the algorithm is likely more complex than the alternative and may require more steps even when no other thread is accessing the data structure.

This brings us to another downside of lock-free and wait-free code: although it can increase the potential for concurrency of operations on a